DAY 33 | HANDS-ON ACTIVITIES

LLMOps: Problem Statements

Seven problems. Every notebook cell runs offline. Prices in the notebook are placeholders, not real prices.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Read the Incident Logs

Level: Warm-up | **Concepts:** log analysis

Using the notebook's simulated week of LLM logs, find when errors spiked, which error code dominated, and when tokens per request jumped.

Sample run / expected result

```
Errors peak 10:00 to 11:00 on 2026-12-02, mostly 429
Tokens per request triple from 2026-12-04
```

Hints

- Group by hour and status; plot tokens per request by day.

Problem 2: Backoff with Jitter

Level: Easy | **Concepts:** retries

Wrap the flaky provider in a retry function with exponential backoff and jitter. Measure success rate with 0, 2 and 4 retries.

Sample run / expected result

```
retries=0: 70% | retries=2: 98% | retries=4: 100%
```

Hints

- Retry only transient errors, never 400 Bad Request.

Problem 3: Fallback and Circuit Breaker

Level: Medium | **Concepts:** resilience patterns

Add a fallback model and a circuit breaker that opens after 3 consecutive failures. Show the user always gets a response.

Sample run / expected result

```
breaker OPEN -> served by fallback
after cool-down -> HALF-OPEN -> CLOSED
```

Hints

- Track consecutive failures and the time the breaker opened.

Problem 4: AgentOps Manifest NEW (v4)

Level: Medium | **Concepts:** versioning prompts, tools, skills, MCP, models

Build a release manifest for the SmartDesk agent with a short hash for every prompt, tool schema, skill file and MCP server version. Change one tool description and show which part of the manifest changed.

Sample run / expected result

```
changed parts: ['tools.refund_tool']
manifest 2.3.0 -> 2.3.1
```

Hints

- hashlib.sha256(text.encode()).hexdigest()[:6]
- Hash the tool schema as sorted JSON.

Problem 5: Model Deprecation Drill NEW (v4)

Level: Medium | **Concepts:** deprecation runbook

Given a list of manifests and a deprecation notice for one model, list every affected release, run the golden set on the candidate model (offline stand-in), and write the runbook decision.

Sample run / expected result

```
affected: ['smartdesk-agent-2.3.0', 'faq-bot-1.1.0']
candidate pass rate 0.92 >= 0.90 -> CANARY
```

Hints

- Use the approved model list in docs/MODEL_GUIDE.md as the source of candidates.

Problem 6: Cost per Feature and Two Model Choices

Level: Medium | **Concepts:** token accounting, placeholders

From the simulated request log, compute cost per feature, then compare the monthly cost of serving the FAQ bot with the large and the small model using placeholder prices.

Sample run / expected result

```
agent_research: 45% of cost from 9% of requests
faq_bot all-small: 6% of all-large cost (placeholder prices)
```

Hints

- Look for features with long contexts or many calls per request.

Problem 7: Routing Trade-off

Level: Stretch | **Concepts:** cost vs quality, latency vs accuracy

Route easy questions to the small model and hard ones to the large model. Compare cost and simulated quality for all-large, all-small and routed, then write a one-paragraph recommendation.

Sample run / expected result

```
all large: cost 100%, quality 0.921
routed: cost 34%, quality 0.914 (simulated quality)
```

Hints

- Validate routing on your golden set before trusting it.